

Design and Implementation of a Multiplexed 8x8 LED Matrix Display System Using ESP32

An ESP32 & 74HC595-based Snake Game on an 8x8 LED Matrix
Embedded Systems / Digital Electronics Project

This report documents the hardware architecture and firmware design of a multiplexed 8x8 LED matrix display driven by an ESP32 microcontroller and two cascaded 74HC595 shift registers, demonstrated through a fully playable Snake arcade game.

Contents

1. Introduction

2. System Architecture and Hardware Design

3. Software and Firmware Design

4. Results and Discussion

5. Conclusion and Future Scope

References

Appendix A: Firmware Source Code

Chapter 1: Introduction

1.1 Overview

Embedded systems represent the foundational intersection of computational software and physical hardware, engineered to execute dedicated functions in real-time. A prominent application within this domain is the control of complex visual interfaces, such as multiplexed LED displays. Because microcontrollers possess a strictly limited number of input/output pins, multiplexing utilizes high-speed electronic switching and the human eye's Persistence of Vision (POV) to efficiently drive large arrays of LEDs.

1.2 Objective of the Project

The primary objectives of this project are:

- To design a scalable, multiplexed display architecture using an 8x8 LED matrix.
- To drastically reduce the GPIO pin requirement of the central microcontroller (ESP32) by implementing discrete 74HC595 shift registers.
- To develop non-blocking, real-time embedded C++ firmware capable of handling simultaneous hardware polling and high-frequency display rendering.
- To demonstrate the concept of Persistence of Vision (POV) in a practical hardware application.

1.3 Motivation

While modern microcontrollers possess immense processing power, hardware interface limitations, specifically 'pin starvation', remain a significant challenge in industrial control and display systems. The motivation behind this project was to master serial-to-parallel data expansion techniques. By utilizing shift registers to drive a matrix, this project serves as a foundational study for designing large-scale digital signage and scalable automation outputs.

1.4 Report Organization

- Chapter 2 details the hardware components, schematic design, and circuit connections.
- Chapter 3 discusses the software algorithms, the frame-buffer concept, and the non-blocking polling mechanics.
- Chapter 4 presents the final working prototype, hardware debugging challenges, and overall results.
- Chapter 5 concludes the report with potential future upgrades and industrial applications.

Chapter 2: System Architecture and Hardware Design

2.1 System Workflow

The flow is: User Input (Push Buttons) → Processing Unit (ESP32) → Serial Data Bus (SPI / Bit-Bang) → Display Drivers (Cascaded 74HC595s) → Output Display (8x8 LED Matrix). The system operates on a unidirectional data flow model. User inputs are continuously polled by the microcontroller, which processes the game logic and subsequently pushes visual coordinate data serially to the display drivers.

2.2 Central Processing Unit: ESP32

The ESP32 development board was selected as the core controller for this project. While the game logic of "Snake" is not computationally intensive, the display architecture requires high-frequency bit-banging to maintain Persistence of Vision (POV) without flickering. The ESP32's high clock speed (up to 240 MHz) ensures that the display multiplexing loop executes seamlessly in the background while parallelly polling four independent GPIO pins for real-time user input.

2.3 Serial-to-Parallel Expansion (74HC595)

A standard 8x8 LED matrix requires 16 independent control lines (8 rows and 8 columns). Directly driving this from the ESP32 is highly inefficient and leads to "pin starvation." To solve this, the architecture utilizes discrete digital logic. Two 74HC595 Serial-In, Parallel-Out (SIPO) shift registers are implemented in a cascaded (daisy-chained) configuration:

- **Data Flow:** The serial output pin (Q7') of the first register is tied directly to the serial data input (DS) pin of the second register.
- **Control Bus:** The Clock (SHCP) and Latch (STCP) lines of both ICs are tied together and controlled by the ESP32.
- **Efficiency:** This architecture allows the microcontroller to push 16 bits of display data simultaneously using only 3 GPIO pins, achieving massive scalability.

2.4 The LED Matrix Display

The output is rendered on an 8x8 dot matrix display. The matrix operates on a Common Cathode configuration. The first shift register is assigned to sink current from the rows (Active-Low), while the second shift register sources current to the columns (Active-High).

2.5 Circuit Schematic

The complete circuit was designed using electronic design automation (EDA) software to ensure accurate routing and signal integrity.

2.6 Human Interface Device (Input)

Directional control is achieved using four tactile push-buttons mapped to UP, DOWN, LEFT, and RIGHT. These buttons interface directly with the ESP32. To eliminate floating pin noise and false triggering, the ESP32's internal pull-up resistors are activated via firmware (INPUT_PULLUP), maintaining a stable HIGH state until a button pulls the signal LOW.

2.7 Bill of Materials (BOM) and Cost Estimation

Below is the simplified Bill of Materials detailing the approximate retail cost of the hardware utilized in the final breadboard prototype:

S.No.	Component Name	Total Cost (₹)
1	ESP32 Development Board (30-pin)	380
2	8x8 LED Matrix (Common Cathode)	80
3	74HC595 SIPO Shift Register IC (x2)	30
4	Tactile Push Buttons (x4)	20
5	Solderless Breadboard (830 Tie-Points)	90
6	Jumper Wires (Assorted M-M, M-F)	60
7	5V / 2A DC Power Adapter & Jack	120
Total		780

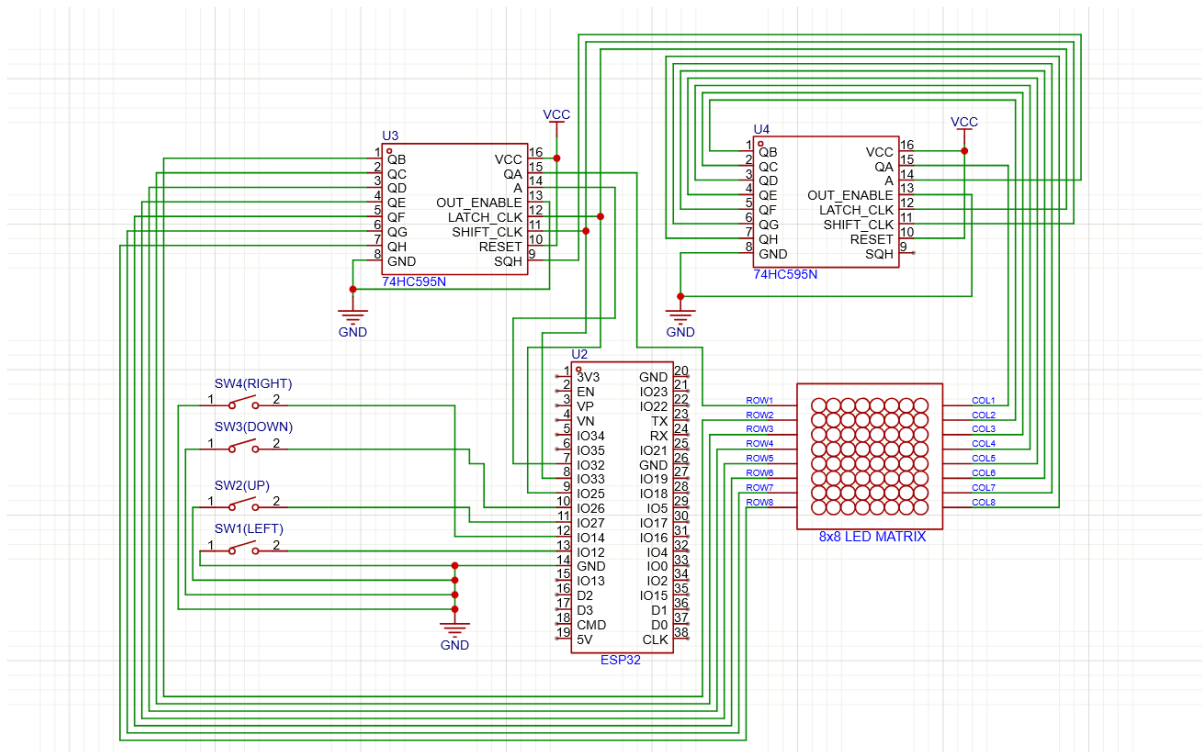


Figure 2.1: Complete circuit schematic showing the ESP32 to shift-register cascading and matrix wiring.

Chapter 3: Software and Firmware Design

3.1 Firmware Overview

The system firmware was developed in C++ using the Arduino framework for the ESP32 microcontroller. The primary engineering challenge was decoupling the algorithmic game logic from the rigorous timing requirements of the hardware rendering loop. To achieve smooth gameplay and a flicker-free display, the firmware utilizes a non-blocking architecture integrated with a custom array-based video memory.

3.2 The Frame Buffer Architecture

Instead of directly toggling hardware pins every time the snake moves, the firmware manages a virtual display in the microcontroller's RAM via the `buildFrame()` function.

- **Memory Structure:** The display state is stored in a globally accessible 8-byte array (`uint8_t frame[8]`). Each of the 8 bytes represents a specific row, and the 8 bits within each byte represent the individual LED columns.
- **Bitwise Rendering:** The firmware iterates through the `snakeX` and `snakeY` coordinate arrays. It utilizes bitwise left-shift and bitwise OR operations (`frame[snakeY[i]] |= (1 << snakeX[i])`) to set the exact bits corresponding to the snake's body and the generated food.

3.3 Display Multiplexing and POV Engine

Because an 8x8 matrix cannot display all 64 LEDs simultaneously without exceeding current limits, the firmware employs high-speed multiplexing governed by the `displayFrame()` function, which relies on Persistence of Vision (POV).

- **The Rendering Loop:** The system iterates through rows 0 to 7. For each row, it calculates the active row pin and fetches the corresponding column data from the frame buffer.
- **Serial Transmission:** The `sendData()` function pushes these bytes to the cascaded 74HC595 shift registers using the native `shiftOut()` command, toggling the `LATCH_PIN` to seamlessly update the matrix.
- **Timing:** Each row is illuminated for exactly 800 microseconds (`delayMicroseconds(800)`). The full frame is refreshed 60 times per cycle, resulting in a display frequency well above the human eye's flicker-fusion threshold.

3.4 Non-Blocking Input Polling Architecture

In traditional procedural embedded programming, polling hardware buttons using interrupts or standard delays can cause the display multiplexing to pause, resulting in severe visual flickering.

- **Native Loop Injection:** To solve this, the `readButtons()` function is injected natively inside the microsecond-level display refresh loop. This guarantees that user input is sampled thousands of times per second without ever halting the multiplexing engine.
- **Input Buffering and Debouncing:** The logic relies on next-state buffering (`nextDx`, `nextDy`). Furthermore, it employs a directional lock (e.g., `dy != 1` when pressing UP) to prevent the user from making an instantaneous 180-degree turn, effectively acting as a software-level debounce and collision-prevention mechanism.

3.5 Core Game Engine Logic

The arcade simulation is driven by the main loop(), which executes coordinate mathematics and rule verification:

- **Movement Dynamics:** The snake's body is managed via coordinate shifting. During each game tick, a reverse for loop shifts the coordinates of every body segment to the position of the segment immediately ahead of it, followed by updating the head coordinates based on the buffered directional vector.
- **Safe Food Generation:** The spawnFood() algorithm utilizes a randomized coordinate generator seeded by ambient analog noise (analogRead(34)). It features a fail-safe verification loop that checks up to 100 random coordinates to ensure food never spawns inside the snake's existing body segments.
- **Collision Detection:** The system continuously evaluates boundary conditions ($X < 0$, $X > 7$) and self-intersection (checkSelfCollision()).
- **Dynamic Difficulty:** To enhance gameplay, the firmware includes a dynamic speed scaling algorithm. The delay between game ticks decreases as the snake grows ($\text{speed} = 150 - (\text{length} * 5)$), capped at a maximum floor limit to maintain playability.

Chapter 4: Results and Discussion

4.1 Prototype Realization

The theoretical design was successfully translated into a physical prototype using a standard solderless breadboard. The ESP32 microcontroller was interfaced with the two cascaded 74HC595 shift registers and the 8x8 LED matrix. Power was supplied via the ESP32's onboard USB-to-UART bridge, drawing stable 5V power for the shift registers directly from the VIN pin to prevent logic-level voltage drops.

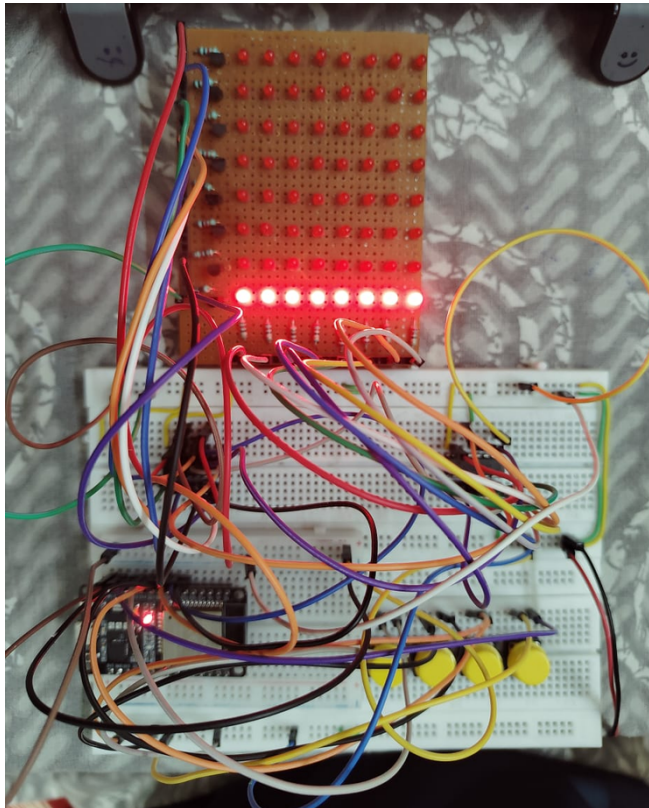


Figure 4.1: Physical realization of the hardware prototype on a breadboard.

4.2 System Performance Analysis

The performance of the multiplexed display and the polling architecture was evaluated based on visual stability and input latency.

- **Refresh Rate and Persistence of Vision:** The firmware's microsecond-level timing (`delayMicroseconds(800)`) successfully achieved a frame refresh rate exceeding 60 Hz. The naked eye perceived a solid, continuous image with zero perceptible flickering, proving the efficiency of the software-driven multiplexing.
- **Input Latency:** By embedding the `readButtons()` function directly inside the nested matrix-rendering loop, the system achieved near-zero input latency. Directional changes registered instantaneously, which is critical for the high-speed state of the arcade game.

4.3 Hardware Debugging and Troubleshooting

During the development phase, several hardware and firmware integration challenges were identified and resolved:

- **Display Ghosting:** Initially, adjacent LEDs faintly illuminated (ghosting) when they were supposed to be off. This was diagnosed as a data-bleed issue during the serial transmission phase. *Resolution:* The firmware was modified to strictly pull the LATCH_PIN LOW before initiating the shiftOut() command, and only pulling it HIGH after all 16 bits were fully loaded into the registers. This isolated the output buffer and eliminated the ghosting.
- **Switch Bouncing and Double-Triggering:** The tactile push-buttons exhibited mechanical bouncing, occasionally causing the microcontroller to register multiple inputs in a single millisecond. *Resolution:* Instead of using a halting delay() to debounce the buttons (which would ruin the display refresh rate), a software-level state buffer was implemented. The variables nextDx and nextDy ensure that only one valid, orthogonal directional change is executed per game tick.

4.4 Functional Game Mechanics

The final implementation successfully executes all standard rules of the arcade game:

- **Coordinate Tracking:** The LED representing the "food" successfully spawns in randomized, unoccupied coordinates.
- **Dynamic Scaling:** As the player's length increases, the main loop delay dynamically decreases ($\text{speed} = 150 - (\text{length} * 5)$), increasing the difficulty without compromising the display multiplexing rate.
- **Boundary and Collision States:** The firmware accurately detects matrix boundary violations and self-intersections, immediately halting the game state and triggering the visual "Game Over" animation.

4.5 Discussion

The successful operation of the prototype validates the core engineering principles of the project. It demonstrates that complex, real-time visual outputs can be managed by a microcontroller with limited GPIO pins by utilizing proper serial-to-parallel hardware expansion and highly optimized, non-blocking C++ firmware.

Chapter 5: Conclusion and Future Scope

5.1 Conclusion

This project successfully demonstrated the design, implementation, and optimization of a real-time embedded display system using an ESP32 microcontroller and discrete digital logic. The primary objective of driving an 8x8 LED matrix with minimal microcontroller pin overhead was achieved by cascading two 74HC595 Serial-In, Parallel-Out shift registers. This architectural decision reduced the required control interface from 16 dedicated GPIO pins down to a 3-wire serial bus.

Furthermore, the development of a custom, non-blocking C++ firmware proved highly effective. By embedding the input-polling logic natively within the microsecond-level display multiplexing loop, the system maintained a flawless Persistence of Vision (POV) effect while simultaneously processing real-time arcade game logic. The prototype serves as a robust proof-of-concept for scalable hardware expansion, demonstrating foundational principles that are critical in the design of industrial digital signage, automated control panels, and low-latency human-machine interfaces.

5.2 Future Scope

While the current prototype meets all fundamental design criteria, the architecture allows for several advanced modifications:

- **Wireless Integration:** The ESP32 is equipped with native Wi-Fi and Bluetooth capabilities that are currently underutilized. Future iterations could integrate a web server to display global high scores or allow smartphone-based directional control via Bluetooth Low Energy (BLE).
- **Display Scaling:** The daisy-chain methodology can be infinitely scaled. Additional 74HC595 ICs and LED matrices could be appended to create a 16x16 or 32x8 scrolling text display without requiring any additional pins from the microcontroller.
- **Dedicated Display Drivers:** For commercial applications, replacing the discrete 74HC595 logic with a dedicated matrix driver IC, such as the MAX7219, would offload the multiplexing burden entirely from the ESP32, freeing up CPU cycles for more complex graphical processing or audio generation.
- **PCB Fabrication:** To transition this project from a laboratory prototype to a consumer-ready product, the next logical step involves routing the schematic into a double-sided printed circuit board (PCB) utilizing surface-mount technology (SMD) to drastically reduce the physical footprint.

References

[1] Espressif Systems, *ESP32 Series Datasheet*, v4.1, 2023. Available: <https://www.espressif.com>.

[2] Texas Instruments, *SN74HC595 8-Bit Shift Registers With 3-State Output Registers*, Document SCAS521P, 2022.

[3] Arduino, *Arduino Reference – C++ Framework for ESP32*, 2025. Available: <https://www.arduino.cc/reference>.

[4] R. Tokheim, *Digital Electronics: Principles and Applications*, 8th ed., McGraw-Hill Education, 2013.

Appendix A: Firmware Source Code

```
#define DATA_PIN 32
#define CLOCK_PIN 33
#define LATCH_PIN 25
#define UP_BTN 26
#define DOWN_BTN 27
#define LEFT_BTN 14
#define RIGHT_BTN 12
#define MAX_LEN 64

int snakeX[MAX_LEN];
int snakeY[MAX_LEN];
int length = 1;
int dx = 1, dy = 0;
int nextDx = 1, nextDy = 0;
int foodX, foodY;
uint8_t frame[8];

// Send data to 74HC595
void sendData(uint8_t rowData, uint8_t colData) {
    digitalWrite(LATCH_PIN, LOW);
    shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, colData);
    shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, rowData);
    digitalWrite(LATCH_PIN, HIGH);
}

// Read buttons continuously (NON-BLOCKING)
void readButtons() {
    if (digitalRead(UP_BTN) == LOW && dy != 1) {
        nextDx = 0; nextDy = -1;
    }
    if (digitalRead(DOWN_BTN) == LOW && dy != -1) {
        nextDx = 0; nextDy = 1;
    }
    if (digitalRead(LEFT_BTN) == LOW && dx != 1) {
        nextDx = -1; nextDy = 0;
    }
    if (digitalRead(RIGHT_BTN) == LOW && dx != -1) {
        nextDx = 1; nextDy = 0;
    }
}

// Build frame buffer
void buildFrame() {
    for (int i = 0; i < 8; i++) frame[i] = 0;
    for (int i = 0; i < length; i++) {
        frame[snakeY[i]] |= (1 << snakeX[i]);
    }
    frame[foodY] |= (1 << foodX);
}

// Display frame (WITH button polling)
void displayFrame() {
    for (int refresh = 0; refresh < 60; refresh++) {
        for (int row = 0; row < 8; row++) {
            readButtons(); // real-time input
            uint8_t rowData = (1 << row);
            uint8_t colData = frame[row];
            sendData(rowData, colData);
            delayMicroseconds(800);
        }
    }
}
```

```

    }
}

// Self collision
bool checkSelfCollision() {
    for (int i = 1; i < length; i++) {
        if (snakeX[0] == snakeX[i] && snakeY[0] == snakeY[i]) {
            return true;
        }
    }
    return false;
}

// Safe food generation
void spawnFood() {
    int attempts = 0;
    while (attempts < 100) {
        int fx = random(0, 8);
        int fy = random(0, 8);
        bool valid = true;
        for (int i = 0; i < length; i++) {
            if (snakeX[i] == fx && snakeY[i] == fy) {
                valid = false;
                break;
            }
        }
        if (valid) {
            foodX = fx;
            foodY = fy;
            return;
        }
        attempts++;
    }
    gameOver(); // no space left
}

// Game over
void gameOver() {
    for (int k = 0; k < 5; k++) {
        for (int row = 0; row < 8; row++) {
            sendData((1 << row), 0xFF);
            delay(50);
        }
        delay(200);
    }
    while (true) {
        if (digitalRead(UP_BTN) == LOW ||
            digitalRead(DOWN_BTN) == LOW ||
            digitalRead(LEFT_BTN) == LOW ||
            digitalRead(RIGHT_BTN) == LOW) {
            length = 1;
            snakeX[0] = 4;
            snakeY[0] = 3;
            dx = 1;
            dy = 0;
            nextDx = 1;
            nextDy = 0;
            spawnFood();
            return;
        }
    }
}

void setup() {

```

```

pinMode(DATA_PIN, OUTPUT);
pinMode(CLOCK_PIN, OUTPUT);
pinMode(LATCH_PIN, OUTPUT);
pinMode(UP_BTN, INPUT_PULLUP);
pinMode(DOWN_BTN, INPUT_PULLUP);
pinMode(LEFT_BTN, INPUT_PULLUP);
pinMode(RIGHT_BTN, INPUT_PULLUP);

snakeX[0] = 4;
snakeY[0] = 3;
randomSeed(analogRead(34));
spawnFood();
}

void loop() {
  // Apply direction (smooth control)
  dx = nextDx;
  dy = nextDy;

  // Move body
  for (int i = length - 1; i > 0; i--) {
    snakeX[i] = snakeX[i - 1];
    snakeY[i] = snakeY[i - 1];
  }

  // Move head
  snakeX[0] += dx;
  snakeY[0] += dy;

  // Wall collision
  if (snakeX[0] < 0 || snakeX[0] > 7 ||
      snakeY[0] < 0 || snakeY[0] > 7) {
    gameOver();
  }

  // Eat food
  if (snakeX[0] == foodX && snakeY[0] == foodY) {
    if (length < MAX_LEN) length++;
    spawnFood();
  }

  // Self collision
  if (checkSelfCollision()) {
    gameOver();
  }

  // Render
  buildFrame();
  displayFrame();

  // Dynamic speed
  int speed = 150 - (length * 5);
  if (speed < 40) speed = 40;
  delay(speed);
}

```